

DATABASE MANAGEMENT SYSTEM

Product and Category Management System

Student Name	Balaji MJ
Register Number	TU-6253210211004
Week	Week 2
Subject	Database Management System
Task Name	Product and Category Management System

Description

1. Design tables for Product and Category entities. 2. Define primary and foreign key relationships. 3. Store product details such as name, category, price, and stock. 4. Perform product insertion, updating, and deletion. 5. Generate category-wise product reports.

1. Project Objective

The objective is to create a relational database for managing product and category information. The system connects products with categories through a foreign-key relationship and supports product insertion, updating, deletion, and category-wise reporting.

The database structure is designed to maintain data integrity while allowing product details such as name, category, price, and stock to be stored and queried efficiently.

2. Database and Category Table

A database named ProductCategoryDB can be used for the project. The Category table stores unique category records.

```
CREATE DATABASE ProductCategoryDB;
USE ProductCategoryDB;

CREATE TABLE Category (
    category_id INT PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL
);
```

The category_id column uniquely identifies each category. The NOT NULL constraint ensures that every category has a name.

3. Product Table and Relationship

The Product table stores product name, category, price, and stock. The category_id foreign key connects each product to a category.

```
CREATE TABLE Product (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(100) NOT NULL,  
    category_id INT,  
    price DECIMAL(10,2),  
    stock INT,  
    FOREIGN KEY (category_id)  
        REFERENCES Category(category_id)  
);
```

The relationship is one-to-many: one category can contain multiple products, while each product is associated with a category through category_id.

4. Insert Category and Product Data

Sample category and product records can be inserted to demonstrate the database operations.

```
INSERT INTO Category
(category_id, category_name)
VALUES
(1, 'Electronics'),
(2, 'Accessories'),
(3, 'Home Appliances'),
(4, 'Gaming');

INSERT INTO Product
(product_id, product_name, category_id, price, stock)
VALUES
(101, 'Laptop', 1, 55000.00, 10),
(102, 'Mobile Phone', 1, 25000.00, 20),
(103, 'Keyboard', 2, 1500.00, 35),
(104, 'Mouse', 2, 800.00, 50),
(105, 'Headphones', 2, 2000.00, 25),
(106, 'Gaming Console', 4, 45000.00, 8);
```

5. Product Insertion

New products can be added by inserting a new record into the Product table with a unique product ID, product name, category ID, price, and stock quantity.

```
INSERT INTO Product
(product_id, product_name, category_id, price, stock)
VALUES
(107, 'Smart Watch', 1, 6000.00, 15);
```

The foreign-key constraint ensures that the supplied category_id corresponds to an existing category.

6. Product Updating

Product information can be modified when prices or stock levels change.

```
UPDATE Product
SET price = 58000.00,
    stock = 12
WHERE product_id = 101;
```

The UPDATE operation changes the selected product without affecting unrelated product records.

7. Product Deletion

Products can be removed from the Product table when they are no longer maintained. The selected product is identified using its primary key.

```
DELETE FROM Product  
WHERE product_id = 107;
```

If a product is referenced by another table in an expanded database design, the foreign-key dependency must be considered before deletion.

8. Category-Wise Product Reports

A JOIN query can combine category and product information to generate a category-wise product report.

```
SELECT
    Category.category_name,
    Product.product_id,
    Product.product_name,
    Product.price,
    Product.stock
FROM Category
JOIN Product
    ON Category.category_id = Product.category_id
ORDER BY Category.category_name,
    Product.product_name;
```

The report displays each category together with its products, prices, and available stock.

9. Category Summary Report

Aggregate functions can be used to summarize the number of products and total stock available in each category.

```
SELECT
    Category.category_name,
    COUNT(Product.product_id) AS product_count,
    SUM(Product.stock) AS total_stock,
    AVG(Product.price) AS average_price
FROM Category
LEFT JOIN Product
    ON Category.category_id = Product.category_id
GROUP BY Category.category_id,
    Category.category_name
ORDER BY Category.category_name;
```

This report provides category-level product count, total stock, and average product price.

10. Database Relationships, Expected Output and Conclusion

Table	Primary Key	Foreign Key	Purpose
Category	category_id	None	Stores product categories
Product	product_id	category_id → Category	Stores product details

Expected Results:

- Categories and products are stored in separate related tables.
- Product records contain name, category, price, and stock.
- Products can be inserted, updated, and deleted.
- Foreign-key relationships maintain category consistency.
- Category-wise product and summary reports can be generated.

Conclusion

The Week-2 DBMS Product and Category Management System demonstrates relational database design using primary and foreign keys. It supports complete product data management and provides category-wise reporting using SQL JOIN and aggregate functions.